

VIT BHOPAL UNIVERSITY

Kothri Kalan, Sehore, Madhya Pradesh

PROJECT REPORT

on

RESTAURANT BILLING SYSTEM

(Python Command-Line Application)

Submitted by

Name	Akshat Farkiya
Registration No.	26MIM10154
Branch	Integrated M.Tech in Artificial Intelligence
University	VIT Bhopal University

Submitted to: Mr.BALAJI

Academic Year: 2026 – 2027

Declaration

I, Akshat Farkiya, Registration No. 26MIM10154, a student of Integrated M.Tech in Artificial Intelligence at VIT Bhopal University, hereby declare that the project report titled “Restaurant Billing System” is my original work. It has been carried out as part of my academic curriculum and has not been submitted elsewhere for any other degree or diploma.

Akshat Farkiya

26MIM10154

Date: 29 September 2026

Acknowledgement

I would like to express my sincere gratitude to my faculty members and to VIT Bhopal University for providing the guidance, resources and learning environment that made this project possible. I also thank my friends and family for their constant support and encouragement throughout its development.

Abstract

The Restaurant Billing System is a Python-based command-line application that automates the process of taking orders and generating bills in a small restaurant or café. The system maintains a menu of 27 vegetarian items with prices in Indian Rupees, allows the user to add or remove items from a cart, validates all inputs, and produces an itemised bill that includes 5% GST.

The project uses core Python concepts such as dictionaries, functions, loops, conditional statements, string handling and input validation. It replaces error-prone manual billing with a fast, accurate and easy-to-use program, and provides a strong base for future extensions such as a graphical interface, database storage and online ordering.

1. Introduction

Small restaurants and cafés often prepare bills by hand or with basic calculators. This is slow and prone to arithmetic mistakes, especially during peak hours. A simple software solution can remove these problems by storing prices, calculating totals and applying taxes automatically.

This project implements such a solution as a console application in Python. The user types the name of a menu item, enters a quantity, and the program keeps track of everything in a cart. When the order is complete, a formatted bill with subtotal, GST and grand total is printed.

1.1 Problem Statement

Manual billing in restaurants is time-consuming and error-prone. There is a need for a lightweight, easy-to-use system that maintains a price list, manages an order cart and generates accurate bills including tax.

1.2 Objectives

- To design a menu-driven billing program using Python.
- To allow adding and removing items with quantity control.
- To validate user input and prevent invalid entries.
- To calculate subtotal, GST (5%) and the final payable amount automatically.
- To produce a clear, readable bill for the customer.

1.3 Scope

The system is intended for small eateries, canteens and cafés with a fixed menu. It works on a single terminal and handles one order at a time. It does not currently store orders permanently or support multiple users.

2. Requirements

2.1 Software Requirements

Component	Details
Programming Language	Python 3.x
Operating System	Windows / Linux / macOS
Editor / IDE	VS Code, IDLE, PyCharm or any text editor
External Libraries	None (standard Python only)

2.2 Hardware Requirements

- Any computer capable of running Python 3.
- Minimum 2 GB RAM and negligible disk space.

3. System Design

3.1 Menu Structure

The menu is stored as a Python dictionary in which each key is an item name and each value is its price in rupees. The 27 items fall into the following categories:

Category	Sample Items	Price Range (₹)
Burgers & Snacks	Veg Burger, Cheese Burger, French Fries	100 – 150
Pizza & Noodles	Veg Pizza, Paneer Pizza, Veg Noodles, Hakka Noodles	140 – 280
Starters	Chilli Paneer, Veg Manchurian, Paneer Tikka	180 – 240
Sandwiches & Dosa	Masala Dosa, Paneer Sandwich, Veg Sandwich	100 – 140
Main Course	Veg/Paneer Biryani, Dal Tadka, Paneer Butter Masala	150 – 220
Breads & Rice	Butter Naan, Tandoori Roti, Plain Rice	30 – 100
Beverages	Cold Coffee, Lemon Soda, Virgin Mojito, Fresh Lime Juice, Masala Tea, Coffee, Mineral Water	30 – 120

3.2 Data Structures Used

- menu (dictionary): maps item name to price.
- cart (dictionary): maps item name to the quantity ordered.
- TAX_RATE (constant): fixed GST rate of 0.05 (5%).

3.3 Program Flow

The program begins by displaying the menu and the list of commands. It then enters an infinite loop that reads a command or item name from the user, converts it to title case, and acts on it:

- Item name: asks for quantity and adds the item to the cart.
- remove: shows the cart and reduces or deletes the chosen item.
- cart: displays the current cart contents.
- menu: displays the menu again.
- done: prints the final bill and exits (only if the cart is not empty).
- Anything else: displays an “Item not found” message.

4. Module Description

The program is divided into small functions, each responsible for one task:

Function	Purpose
show_menu()	Displays all 27 menu items with prices.
show_cart()	Lists cart items, quantities and line totals.
get_quantity()	Reads and validates quantity (positive whole number).
add_item(item)	Adds an item to the cart or increases its quantity.
remove_item()	Reduces the quantity of an item or removes it entirely.
print_bill()	Calculates subtotal, 5% GST and grand total, and prints the bill.

4.1 Input Validation

The `get_quantity()` function accepts input as text, strips extra spaces and uses `isdigit()` to make sure it is a whole number. Zero, negative numbers, decimals and text are rejected with a helpful message, so the cart can never contain invalid quantities.

4.2 Bill Calculation

For every cart item the line amount is $\text{price} \times \text{quantity}$. The subtotal is the sum of all line amounts. GST is computed as $\text{subtotal} \times 0.05$ and the grand total is $\text{subtotal} + \text{GST}$. All amounts on the bill are shown with two decimal places.

5. Implementation

The program is implemented in a single Python file. A condensed outline of its structure is given below.

```
TAX_RATE = 0.05
menu = { "Veg Burger": 120, "Cheese Burger": 150, ... }    # 27 items
cart = {}

def show_menu():      # prints all items with prices
def show_cart():      # prints cart items, quantities, line totals
def get_quantity():   # validates quantity input (whole number >= 1)
def add_item(item):   # adds/increments item in cart
def remove_item():    # reduces quantity or deletes item from cart
def print_bill():     # subtotal + 5% GST + grand total

while True:           # main command loop
    order = input("Enter item name or command: ").strip().title()
    if order == "Done": ... print_bill(); break
    elif order == "Remove": ... remove_item()
    elif order == "Cart": ... show_cart()
    elif order == "Menu": ... show_menu()
    elif order in menu: ... add_item(order)
    else: print("Item not found. Check the spelling.")
```

5.1 Sample Output

Example of a cart and final bill for an order of two Veg Burgers and one Cold Coffee:

```
----- YOUR CART -----
Veg Burger      x2   = ₹240
Cold Coffee     x1   = ₹100

===== BILL =====
Veg Burger      x2   = ₹240
Cold Coffee     x1   = ₹100
-----
Subtotal : ₹340.00
GST (5%) : ₹17.00
TOTAL    : ₹357.00
Thank you! Visit again.
```

6. Testing

The program was tested manually with valid and invalid inputs to check that every feature behaves correctly.

Test Case	Input	Expected Result	Status
Add valid item	Veg Burger, qty 2	2 x Veg Burger added	Pass
Add same item again	Veg Burger, qty 1	Quantity becomes 3	Pass
Invalid quantity	abc	Error: enter a valid whole number	Pass
Zero / negative quantity	0	Error: must be at least 1	Pass
Unknown item	Sushi	Item not found message	Pass
Case-insensitive input	veg burger	Item recognised	Pass
Remove item not in cart	Coffee	Not in cart message	Pass
Remove full quantity	Remove 3 of Veg Burger	Item deleted from cart	Pass
Done with empty cart	done	Prompts user to order first	Pass
Bill calculation	2 x Veg Burger (₹240)	GST ₹12.00, Total ₹252.00	Pass

7. Advantages and Limitations

7.1 Advantages

- Simple, fast and easy to use.
- Removes calculation errors in billing.
- Input is case-insensitive and validated.
- Uses only standard Python; no installation of extra packages.
- Clean, modular code that is easy to understand and extend.

7.2 Limitations

- Command-line interface only; no graphical screen.
- Menu is hard-coded and cannot be edited while running.
- Orders and bills are not saved to a file or database.
- Item names must be typed in full; partial names are not supported.
- Does not support discounts, multiple tables or multiple users.

8. Future Scope

- Graphical user interface using Tkinter or a web front end.
- Storing menu and order history in a database such as SQLite or MySQL.
- Saving bills as PDF or printing them on a thermal printer.
- Admin panel to add, edit or delete menu items.

- Support for discounts, coupons and multiple payment methods.
- Daily sales reports and analytics.
- Partial-name search and item numbers for faster ordering.

9. Conclusion

The Restaurant Billing System successfully demonstrates how basic Python programming can solve a real-world problem. It manages a menu, handles an order cart with validation, and produces an accurate itemised bill with GST. The project strengthened my understanding of dictionaries, functions, loops, string handling and error checking, and it provides a solid foundation for building a full-fledged point-of-sale application in the future.

10. References

- Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>
- Sweigart, A. Automate the Boring Stuff with Python. No Starch Press.
- Matthes, E. Python Crash Course. No Starch Press.
- W3Schools. Python Tutorial. <https://www.w3schools.com/python/>